

Higher School of Computer Science

ESI Sidi-Bel-Abbes

Academic Year 2025–2026

Network Digital Twin

with Adversarial Reinforcement Learning

Design and Implementation of an Intelligent Cybersecurity Simulation Platform

Prepared by:

Manel Mostefaoui
Brahmi Sarah Kaouter
Benikhlef Houssam
Anis Boulfoul
Khaled Guessaoui

Supervisor: Mohamed BEDJAOUI

Department of Cybersecurity and Information Systems

ESI Sidi-Bel-Abbes

May 2026

Contents

Abstract	3
1 Introduction	4
1.1 Context and Motivation	4
1.2 Project Objectives	4
1.3 Report Structure	4
2 System Architecture	5
2.1 Overview	5
2.1.1 GNS3 Server — Sarah’s Kali Machine	5
2.1.2 Ubuntu Server VM — Anis’s Machine	6
2.1.3 RL Agents Control Plane — Housseem’s Machine	6
2.1.4 Team Members — ZeroTier Network	6
2.2 External Attack Path	6
3 Network Topology	8
3.1 3-Tier Hierarchical Architecture	8
3.2 Complete IP Address Inventory	9
3.3 VLAN Configuration	9
3.4 OSPF Neighbor Table	10
3.5 Firewall Static Routes	10
3.6 Verified Connectivity Status	10
4 Digital Twin — Monitoring Stack	11
4.1 What is a Digital Twin?	11
4.2 NetBox — Automatic Network Inventory	11
4.2.1 GNS3 → NetBox Auto-Sync	12
4.3 Telegraf — Metrics Collection	12
4.4 InfluxDB — Time-Series Database	13
4.5 Grafana — Security Dashboard	13
4.6 Docker Services Summary	14
4.7 Ansible Automation and Monitoring Integration	14
4.7.1 Overview	14
4.7.2 Managed Devices	14
4.7.3 Playbooks	15
4.7.4 Prometheus Metrics	15
4.7.5 Grafana Dashboard	15
4.7.6 Current Results	16
5 Security Testing	17

5.1	Manual Penetration Testing	17
5.2	Victim VM Configuration	17
5.3	Confirmed Attack Path	17
5.4	Attack Results	18
5.4.1	Attack 1 — Network Reconnaissance (Nmap)	18
5.4.2	Attack 2 — Web Server Access (curl + Nikto)	18
5.4.3	Attack 3 — SSH Brute Force (Hydra)	18
5.4.4	Attack 4 — Post-Exploitation Recon	18
5.4.5	Attack 5 — DoS SYN Flood (hping3)	19
5.5	Manual Testing Summary	19
6	Reinforcement Learning Agents	20
6.1	Introduction to RL in Cybersecurity	20
6.2	Environment Design	21
6.2.1	gns3_connector.py — Network Interface	21
6.2.2	state_collector.py — State Observation	21
6.2.3	attack_executor.py — Attack Simulation Engine	22
6.2.4	defense_executor.py — Defense Simulation Engine	22
6.2.5	environment.py — Main RL Interface	22
6.3	State Space	23
6.4	Action Space	23
6.4.1	Red Agent — Attack Actions (18)	24
6.4.2	Blue Agent — Defense Actions (18)	25
6.5	Kill Chain System	26
6.6	Reward Function	26
6.6.1	Red Agent Reward	27
6.6.2	Blue Agent Reward	27
6.7	Scenario Variety	27
6.8	Training Algorithm — Deep Q-Network (DQN)	28
6.8.1	Neural Network Architecture	28
6.8.2	Target Network	28
6.8.3	Exploration Strategy	29
6.8.4	Experience Replay	29
6.8.5	Hyperparameters Summary	29
6.9	Self-Play Training Loop	30
6.10	Logging and Metrics	30
6.10.1	CSV Episode Log	30
6.10.2	Per-Action Statistics (JSON)	31
6.10.3	TensorBoard Integration	31
6.11	System Architecture Summary	32
6.12	Summary	32
7	Results and Conclusion	34
7.1	Project Achievements	34
7.2	Key Technical Challenges	34
7.3	Limitations	35
7.4	Future Work	35
7.5	Conclusion	35

Abstract

This report presents the design and implementation of a **Network Digital Twin** enhanced with **Adversarial Reinforcement Learning** agents for autonomous cybersecurity simulation. The platform consists of three interconnected layers: a virtual enterprise network emulated in GNS3, a real-time monitoring stack (Telegraf, InfluxDB, Grafana, NetBox), and two competing RL agents — a red attacker and a blue defender .

The attacker agent learns to exploit network vulnerabilities (SSH brute force, HTTP exploitation, DoS attacks) while the defender agent learns to detect and block these intrusions by monitoring live network metrics.

Chapter 1

Introduction

1.1 Context and Motivation

Traditional cybersecurity tools are **rule-based**: they detect known attack patterns but fail against novel or adaptive threats. Meanwhile, enterprise networks are growing in complexity, making manual security monitoring increasingly impractical.

This project addresses both challenges by combining two emerging technologies:

- **Network Digital Twin**: A live virtual replica of a network that mirrors its real-time state, enabling safe testing and analysis without touching production.
- **Adversarial Reinforcement Learning**: Two AI agents that learn attack and defense strategies autonomously through trial and error, discovering strategies that rule-based systems would miss.

1.2 Project Objectives

1. Build a realistic virtual enterprise network in GNS3 (18 nodes, 3-tier architecture)
2. Create a digital twin monitoring stack with real-time metrics visualization
3. Document the network inventory using NetBox with automatic GNS3 synchronization
4. Conduct manual penetration testing to establish baseline attack results
5. Establish a real external attack path via ZeroTier routing through the GNS3 network
6. Train RL attacker and defender agents that operate on the live network
7. Compare manual attack results with RL agent performance

1.3 Report Structure

This report is organized as follows: Chapter 2 describes the system architecture, Chapter 3 details the network topology, Chapter 4 covers the monitoring stack, Chapter 5 presents the security testing results, Chapter 6 explains the RL agent design and training, and Chapter 7 concludes with results and future work.

Chapter 2

System Architecture

2.1 Overview

The platform consists of four interconnected components linked via a private ZeroTier virtual network (Network ID: 743993800ff4423d), allowing all five team members to collaborate remotely as if on the same LAN.

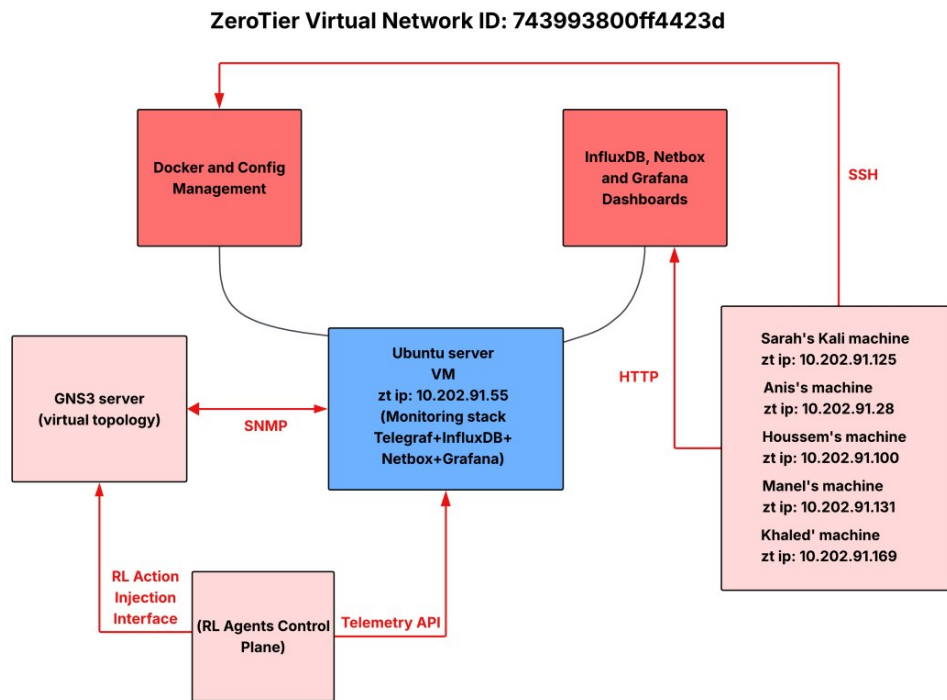


Figure 2.1: Infrastructure Architecture

2.1.1 GNS3 Server — Sarah's Kali Machine

- **ZeroTier IP:** 10.202.91.125
- **Role:** Hosts GNS3 server v3.0.5 on port 3080, runs all 18 virtual network nodes (routers, switches, firewall, PCs, victim VM)

- **Virtualization:** QEMU/KVM for Cisco IOS and Alpine Firewall VMs, Docker for the Victim-1 container
- **Bridge:** virbr0 (192.168.122.0/24) via libvirt connects GNS3 virtual network to the real ZeroTier network
- **API:** GNS3 REST API v3 (<http://10.202.91.125:3080>)

2.1.2 Ubuntu Server VM — Anis’s Machine

- **ZeroTier IP:** 10.202.91.55
- **Role:** Hosts the complete monitoring stack in Docker containers
- **Containers:** Telegraf (SNMP collector), InfluxDB (time-series DB), Grafana (dashboard), NetBox (inventory), PostgreSQL, Redis
- **Data collection:** Telegraf polls GNS3 L3 devices every 60s via SNMP
- **Auto-sync:** `gns3_netbox_sync.py` systemd service automatically syncs GNS3 topology into NetBox

2.1.3 RL Agents Control Plane — Houssem’s Machine

- **ZeroTier IP:** 10.202.91.100
- **Role:** Runs the Python attacker and defender RL agents
- **Action injection:** Pushes attack/defense actions to GNS3 via REST API
- **Telemetry:** Reads live network state from InfluxDB on Anis’s VM

2.1.4 Team Members — ZeroTier Network

All five team members connect via ZeroTier (Network ID: 743993800ff4423d), creating a private virtual LAN over the internet with no router configuration required.

Table 2.1: Team Members and ZeroTier IPs

Member	ZeroTier IP	Role
Sarah Brahmi	10.202.91.125	GNS3 Server host
Anis Boulfoul	10.202.91.28	Ubuntu VM / Monitoring Stack
Houssem Benikhlef	10.202.91.100	RL Agents
Manel Mostefaoui	10.202.91.131	Network Architect / Attacker

2.2 External Attack Path

A key feature of this platform is the ability for external attackers to reach the GNS3 internal network from outside via ZeroTier routing:

Table 2.2: External Attack Path — 5 Hops

Hop	Device	IP
1	Manel’s Kali (attacker)	10.202.91.131
2	Sarah’s laptop (ZeroTier)	10.202.91.125
3	Alpine-Firewall-1	192.168.122.181
4	Router2	10.10.20.2
5	Victim-1 (target)	10.20.20.200

Chapter 3

Network Topology

3.1 3-Tier Hierarchical Architecture

The GNS3 network follows the standard enterprise 3-tier design:

- **Core Layer:** Alpine-Firewall-1 connects to internet (NAT2/Cloud3) and distributes to both router branches
- **Distribution Layer:** Router1 + SwitchL31 (left branch), Router2 + SwitchL32 (right branch), OSPF area 0
- **Access Layer:** Switch1–4 connect end devices (PCs and victim VM) via 802.1q VLANs

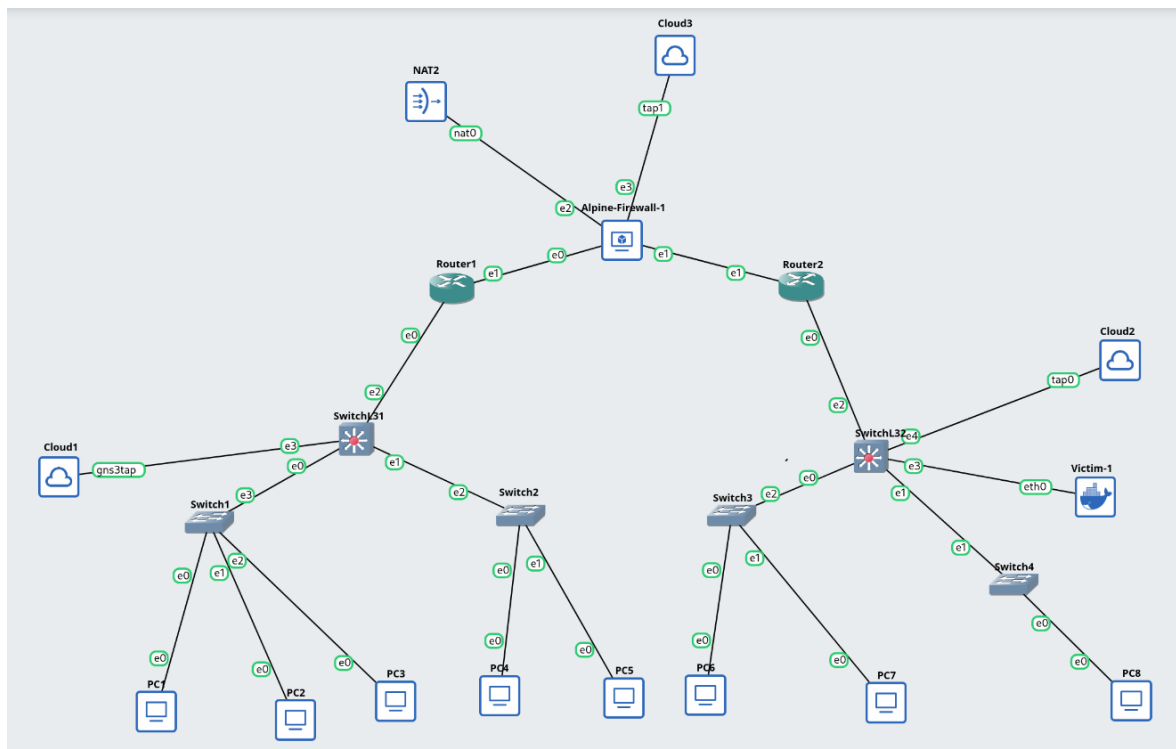


Figure 3.1: GNS3 Network Topology

3.2 Complete IP Address Inventory

Table 3.1: Complete IP Address Table — Verified Live Values

Device	Interface	IP Address	Connected To	Protocol
Alpine-Firewall-1	eth0	10.10.10.1/30	Router1 Gi0/1	Static
Alpine-Firewall-1	eth1	10.10.20.1/30	Router2 Gi0/1	Static
Alpine-Firewall-1	eth2	192.168.122.181/24	NAT2 / Cloud3	NAT
Router1	Gi0/1	10.10.10.2/30	Firewall eth0	Static + OSPF
Router1	Gi0/0	20.0.0.2/30	SwitchL31 Gi0/2	OSPF area 0
Router2	Gi0/1	10.10.20.2/30	Firewall eth1	Static + OSPF
Router2	Gi0/0	192.168.2.1/30	SwitchL32 Gi0/2	OSPF area 0
SwitchL31	Gi0/2	20.0.0.1/30	Router1 Gi0/0	OSPF area 0
SwitchL31	Vlan10 SVI	192.168.10.1/24	Switch1, Switch2	OSPF stub
SwitchL31	Vlan20 SVI	192.168.20.1/24	Switch1, Switch2	OSPF stub
SwitchL31	Gi0/3	172.16.0.1/24	Cloud1 (gns3tap)	Management
SwitchL32	Gi0/2	192.168.2.2/30	Router2 Gi0/0	OSPF area 0
SwitchL32	Vlan30 SVI	192.168.30.1/24	Switch3	OSPF stub
SwitchL32	Vlan40 SVI	192.168.40.1/24	Switch4	OSPF stub
SwitchL32	Vlan50 SVI	10.20.20.1/24	Victim-1, Switch4	OSPF stub
PC1	eth0	192.168.10.10/24	Switch1 Gi0/0	Vlan10
PC2	eth0	192.168.10.11/24	Switch1 Gi0/1	Vlan10
PC3	eth0	192.168.20.11/24	Switch1 Gi0/2	Vlan20
PC4	eth0	192.168.20.10/24	Switch2 Gi0/0	Vlan20
PC5	eth0	192.168.10.12/24	Switch2 Gi0/1	Vlan10
PC6	eth0	192.168.30.10/24	Switch3 Gi0/0	Vlan30
PC7	eth0	192.168.30.11/24	Switch3 Gi0/1	Vlan30
PC8	eth0	192.168.40.10/24	Switch4 Gi0/0	Vlan50
Victim-1	eth0	10.20.20.200/24	SwitchL32 Gi0/3	Vlan50
Cloud2	tap0	10.20.20.254/24	SwitchL32 Gi1/0	TAP / ZeroTier
NAT2	nat0	192.168.122.1	Firewall eth2	NAT / MASQUERADING

3.3 VLAN Configuration

Table 3.2: VLAN Map — Verified Live

VLAN	Name	Subnet	SVI Gateway	Hosts
10	PC1_PC2_PC5	192.168.10.0/24	192.168.10.1 (SwitchL31)	PC1, PC2, PC5
20	PC3_PC4	192.168.20.0/24	192.168.20.1 (SwitchL31)	PC3, PC4
30	VLAN0030	192.168.30.0/24	192.168.30.1 (SwitchL32)	PC6, PC7
40	VLAN0040	192.168.40.0/24	192.168.40.1 (SwitchL32)	—
50	VLAN0050	10.20.20.0/24	10.20.20.1 (SwitchL32)	Victim-1, PC8

3.4 OSPF Neighbor Table

Table 3.3: OSPF Neighbors — Area 0

Router	Router ID	Neighbor	Neighbor ID	State
Router1	1.1.1.1	SwitchL31	3.3.3.1	FULL/DR
Router2	2.2.2.2	SwitchL32	3.3.3.2	FULL/BDR
SwitchL31	3.3.3.1	Router1	1.1.1.1	FULL/BDR
SwitchL32	3.3.3.2	Router2	2.2.2.2	FULL/DR

3.5 Firewall Static Routes

The Alpine Firewall maintains 8 static routes (persistent via `/etc/local.d/routes.start`):

Table 3.4: Alpine-Firewall-1 Static Routes

Destination	Next-hop	Purpose
0.0.0.0/0	192.168.122.1	Default / Internet (NAT2)
192.168.10.0/24	10.10.10.2	Vlan10 (PC1, PC2, PC5)
192.168.20.0/24	10.10.10.2	Vlan20 (PC3, PC4)
192.168.30.0/24	10.10.20.2	Vlan30 (PC6, PC7)
192.168.40.0/24	10.10.20.2	Vlan40
10.20.20.0/24	10.10.20.2	Vlan50 / Victim-1
20.0.0.0/30	10.10.10.2	Router1–SwitchL31 link
192.168.2.0/30	10.10.20.2	Router2–SwitchL32 link

3.6 Verified Connectivity Status

Table 3.5: Full Connectivity Matrix — All Paths Verified

Path	Status	Notes
Firewall ↔ Router1/Router2	Working	Ping success both ways
Router1 ↔ SwitchL31 (OSPF)	FULL	Neighbor 3.3.3.1, DR/BDR elected
Router2 ↔ SwitchL32 (OSPF)	FULL	Neighbor 3.3.3.2, DR/BDR elected
PC1/PC2/PC5 → gateway	Working	192.168.10.1 reachable
PC3/PC4 → gateway	Working	192.168.20.1 reachable
PC6/PC7 → gateway	Working	192.168.30.1 reachable
PC1 → PC6 (cross-network)	Working	Full routed path, ttl=59, ~4ms
Firewall → PC1	Working	Red agent reach confirmed
Firewall → PC6	Working	Via Router2 → SwitchL32
PC1 → Victim-1 (10.20.20.200)	Working	ttl=59, ~2ms
Kali (external) → Firewall	Working	Via ZeroTier, ttl=63, ~13ms
Kali (external) → Victim-1	Working	5 hops, ttl=60, ~15ms
Config persistence after restart	Solved	Startup configs in GNS3 project files
SwitchL31 ↔ SwitchL32	→ Via FW	No direct cross-route by design

Chapter 4

Digital Twin — Monitoring Stack

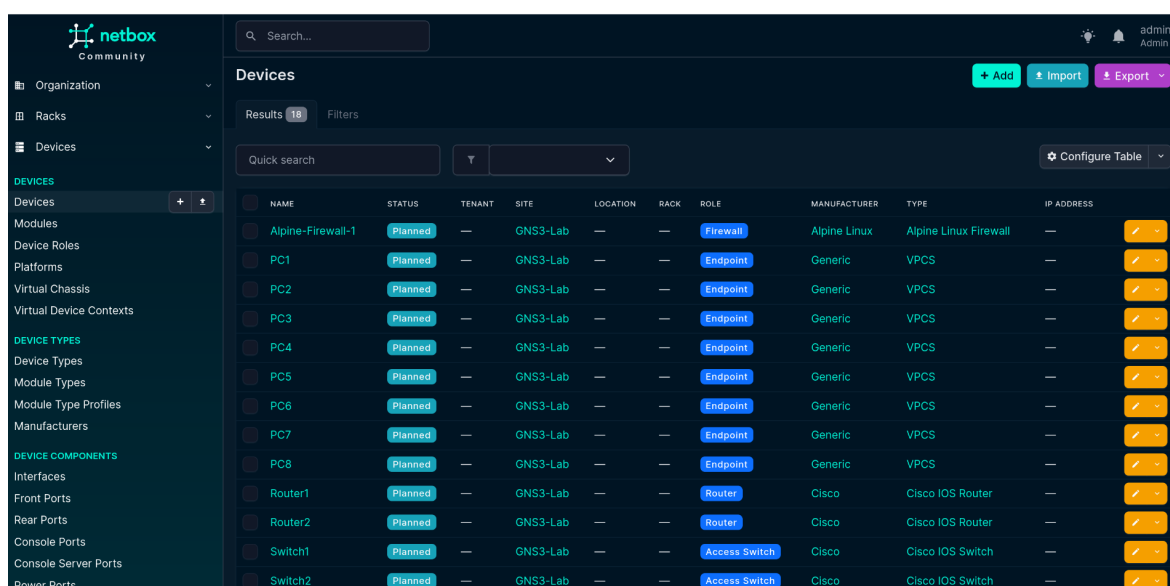
4.1 What is a Digital Twin?

A Network Digital Twin is a live virtual replica of a physical (or virtual) network that mirrors its real-time state. In this project:

- **GNS3 Network** = the “physical” network being twinned
- **NetBox** = structural twin (inventory, topology, IP schema)
- **Telegraf** + **InfluxDB** = behavioral twin (live metrics, traffic, CPU)
- **Grafana** = visual twin (real-time dashboard)

4.2 NetBox — Automatic Network Inventory

NetBox serves as the **source of truth** for the network inventory, documenting all devices, interfaces, and IP addresses. Crucially, NetBox is populated **automatically** via a Python synchronization service that polls the GNS3 API.



NAME	STATUS	TENANT	SITE	LOCATION	RACK	ROLE	MANUFACTURER	TYPE	IP ADDRESS
Alpine-Firewall-1	Planned	—	GNS3-Lab	—	—	Firewall	Alpine Linux	Alpine Linux Firewall	—
PC1	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
PC2	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
PC3	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
PC4	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
PC5	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
PC6	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
PC7	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
PC8	Planned	—	GNS3-Lab	—	—	Endpoint	Generic	VPCS	—
Router1	Planned	—	GNS3-Lab	—	—	Router	Cisco	Cisco IOS Router	—
Router2	Planned	—	GNS3-Lab	—	—	Router	Cisco	Cisco IOS Router	—
Switch1	Planned	—	GNS3-Lab	—	—	Access Switch	Cisco	Cisco IOS Switch	—
Switch2	Planned	—	GNS3-Lab	—	—	Access Switch	Cisco	Cisco IOS Switch	—

Figure 4.1: netbox

4.2.1 GNS3 → NetBox Auto-Sync

The `gns3_netbox_sync.py` script runs as a systemd service on the Ubuntu VM, syncing every 60 seconds:

```

1 # Deploy the sync service
2 sudo cp ~/gns3_netbox_sync.py /opt/gns3_netbox_sync.py
3 sudo cp ~/gns3-netbox-sync.service /etc/systemd/system/
4 sudo systemctl daemon-reload
5 sudo systemctl enable --now gns3-netbox-sync
6
7 # Monitor sync logs
8 journalctl -u gns3-netbox-sync -f

```

Listing 4.1: NetBox Sync Service Deployment

The sync script authenticates with the GNS3 v3 API using OAuth2, fetches live node status, and creates/updates all devices, interfaces, IP addresses, VLANs, and prefixes in NetBox automatically.

Table 4.1: NetBox Inventory Summary

Category	Count	Details
Sites	1	GNS3-Lab
Device Roles	5	Firewall, Router, L3 Switch, Access Switch, Server
Device Types	4	Alpine Firewall, Cisco Router, Cisco L3/L2 Switch
Devices	18	1 firewall, 2 routers, 6 switches, 8 PCs, 1 victim
IP Prefixes	13	5 VLANs + 4 router links + 1 NAT + management
IP Addresses	26	All interfaces documented

Access: <http://10.202.91.55:8080>

4.3 Telegraf — Metrics Collection

Telegraf collects network metrics from all L3 devices every 60 seconds using **SNMP v2c** (Simple Network Management Protocol).

```

1 [agent]
2   interval = "60s"
3   hostname = "monitoring-server"
4
5 [[outputs.influxdb_v2]]
6   urls = ["http://influxdb:8086"]
7   token = "project2026token"
8   organization = "project"
9   bucket = "network"
10
11 [[inputs.snmp]]
12   agents = [
13     "udp://172.16.0.1:161",      # SwitchL31 (via Cloud1 gns3tap)
14     "udp://20.0.0.2:161",      # Router1
15     "udp://192.168.2.1:161",   # Router2
16     "udp://192.168.2.2:161"    # SwitchL32
17   ]
18   community = "public"

```

19

`version = 2`

Listing 4.2: Telegraf SNMP Configuration

4.4 InfluxDB — Time-Series Database

InfluxDB stores all collected metrics in a time-series format optimized for high-frequency network data.

- **URL:** <http://10.202.91.55:8086>
- **Organization:** project
- **Bucket:** network
- **Retention:** 30 days

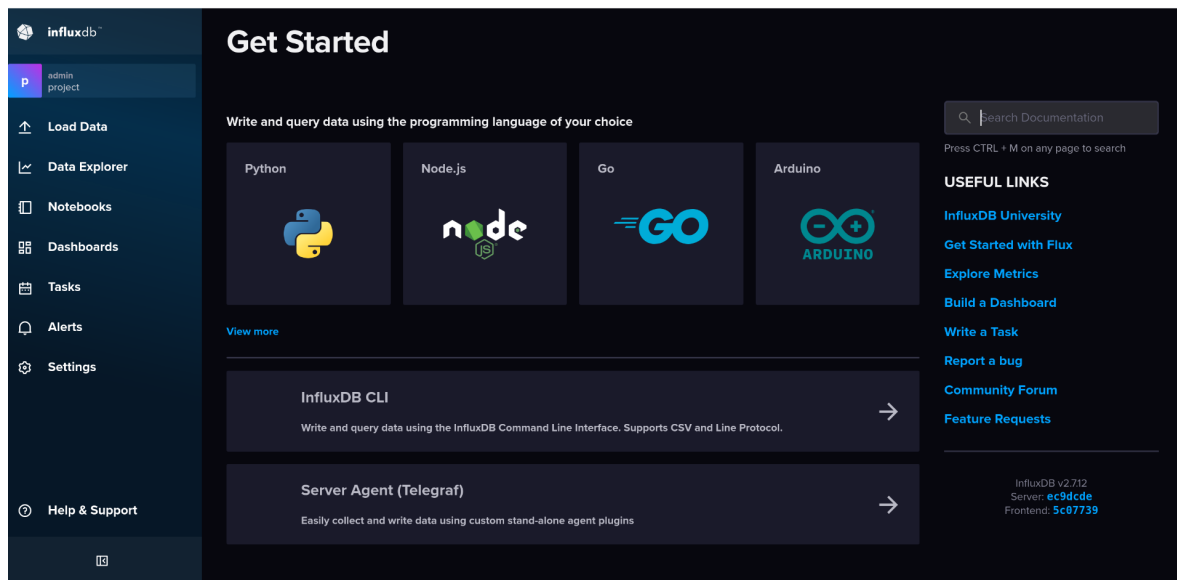


Figure 4.2: influxDB

4.5 Grafana — Security Dashboard

The Grafana dashboard provides real-time visualization of network health and security events.

Table 4.2: Grafana Dashboard Panels

Panel	Type	Metric	Security Purpose
Device CPU Usage	Gauge	snmp.cpu	High CPU = possible DoS
Interface Status	Stat	interface.status	1=UP (green), 2=DOWN (red)
Inbound Traffic	Time Series	interface.in_octets	Traffic spike = attack
Interface Errors	Time Series	interface.in_errors	Error spike = anomaly
Attack vs Defense	Time Series	in + out octets	Agent impact visualization

Access: <http://10.202.91.55:3000>

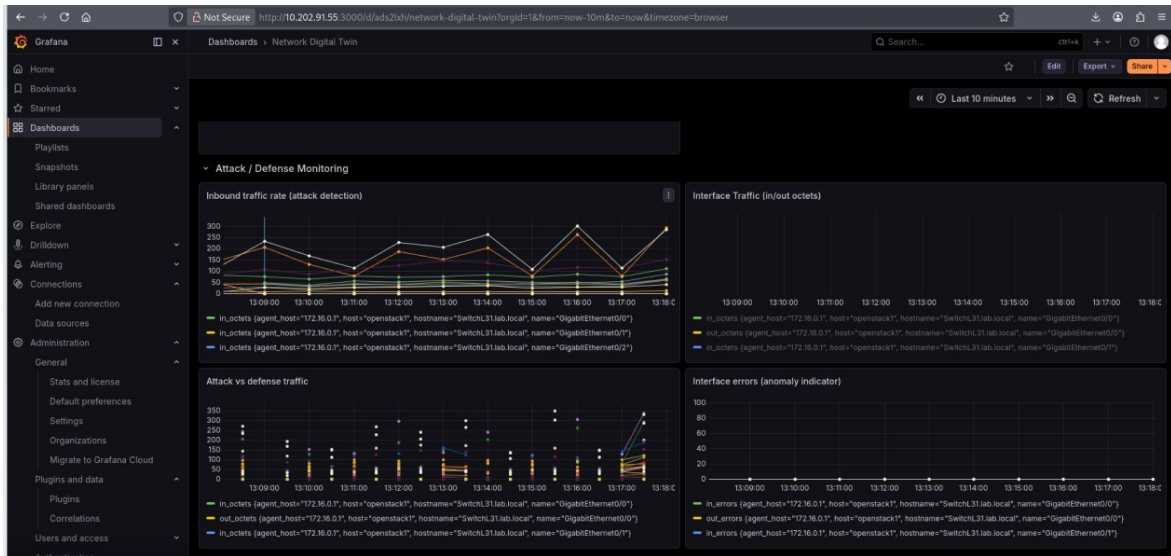


Figure 4.3: Grafana

4.6 Docker Services Summary

Table 4.3: Docker Container Stack

Container	Image	Port	Role	Credentials
telegraf	telegraf:1.29	—	SNMP collector	—
influxdb	influxdb:2.7	8086	Time-series DB	admin/project2026
grafana	grafana/grafana	3000	Dashboard	admin/project2026
netbox	netboxcommunity/netbox	8080	Inventory	admin/Project20262026
netbox-db	postgres:15	5432	NetBox DB	netbox/project2026
netbox-redis	redis:7	6379	NetBox cache	—

4.7 Ansible Automation and Monitoring Integration

4.7.1 Overview

To eliminate manual device inspection, an automation layer was built on the monitoring VM (10.202.91.55) using Ansible, Cron, Prometheus, and Grafana. The system automatically collects configurations, operational state, and backup status from all managed network devices.

4.7.2 Managed Devices

Ansible manages 6 targets:

Table 4.4: Ansible Managed Targets

Target	Type	Connection
router-1	Cisco IOS Router	cisco.ios.ios
router-2	Cisco IOS Router	cisco.ios.ios
switch-l3-1	Cisco IOS L3 Switch	cisco.ios.ios
switch-l3-2	Cisco IOS L3 Switch	cisco.ios.ios
alpine-firewall-1	Alpine Linux	paramiko_ssh
monitoring-vm-services	Ubuntu VM	local

4.7.3 Playbooks

Five playbooks run automatically every day via cron:

Table 4.5: Cron Schedule and Playbooks

Time	Playbook	Purpose
02:00	Reachability Check	Detect unreachable devices
02:05	Configuration Backup	Collect running configs
02:10	Network State Collection	<code>show ip route</code> , OSPF, VLANs, interfaces
02:15	VM Service Backup	Docker, ports, monitoring services
02:20	Backup Status Generation	Generate <code>.prom</code> metrics for Prometheus

4.7.4 Prometheus Metrics

The backup status script generates a `.prom` file scraped by Prometheus via Node Exporter. Key metrics exposed:

```

1 ansible_backup_devices_total           # Total managed targets
2 ansible_backup_devices_present         # Targets with backup files
3 ansible_backup_devices_missing         # Targets missing backups
4 ansible_backup_device_present          # Per-device: 1=exists, 0=missing
5 ansible_backup_file_age_seconds        # Age of each backup file
6 ansible_backup_file_size_bytes         # Size of each backup file

```

Listing 4.3: Ansible Prometheus Metrics

4.7.5 Grafana Dashboard

A dedicated **Ansible Automation Status** dashboard was created in Grafana with panels for total devices, backup coverage, missing backups, Prometheus scrape status, and per-device backup age and file size.

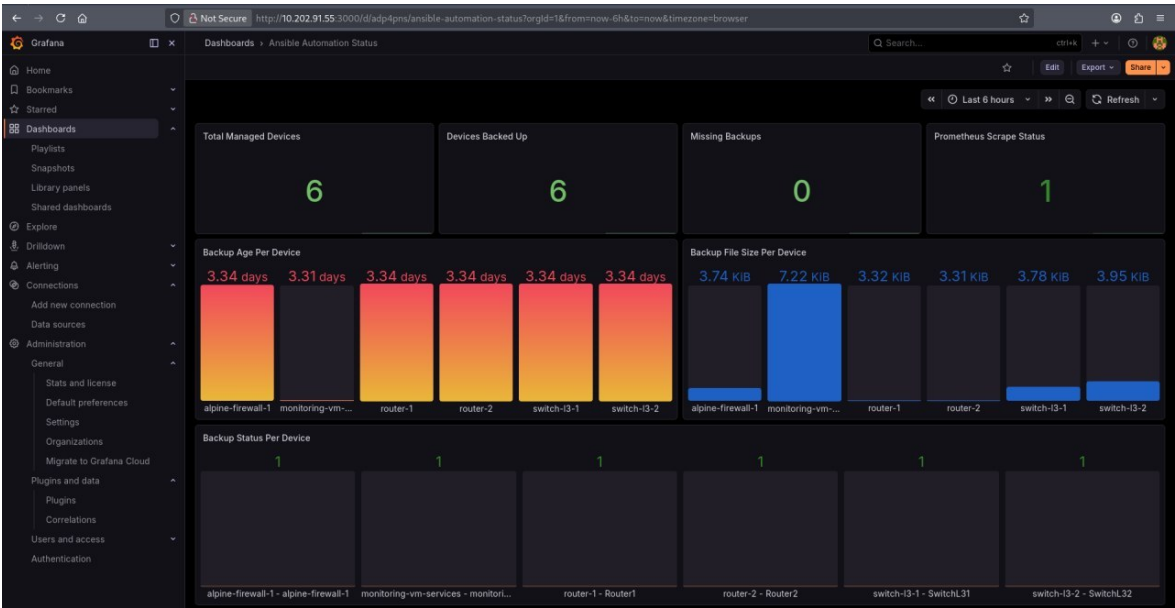


Figure 4.4: Ansible Grafana Dashboard

4.7.6 Current Results

Table 4.6: Automation Status — Verified

Metric	Value
Total managed targets	6
Devices with backups	6
Missing backups	0
Prometheus scraping	Working (1)
Cron scheduling	Active
Grafana dashboard	Working

All 6 managed targets are backed up with zero missing backups, confirming the automation layer is fully operational.

Chapter 5

Security Testing

5.1 Manual Penetration Testing

Before running the RL agents, manual attacks were conducted to establish baseline results and verify that the attack surface was properly configured.

Attack Scenario: External attacker simulation. The attacker (Kali machine at 10.202.91.131) targets Victim-1 (10.20.20.200) from outside the GNS3 network, routing through ZeroTier → Sarah’s laptop → virbr0 → Alpine-Firewall-1 → Router2 → SwitchL32 → Victim-1. The full 5-hop path was confirmed via traceroute.

5.2 Victim VM Configuration

Table 5.1: Victim VM Specifications

Property	Value
IP Address	10.20.20.200/24
Gateway	10.20.20.1 (SwitchL32 Vlan50 SVI)
Connection	SwitchL32 Gi0/3, Vlan50
OS	Ubuntu 22.04 LTS (Docker container)
Kernel	6.19.11+kali-amd64
SSH Service	OpenSSH (port 22)
Web Service	Apache 2.4.52 (port 80)
Weak User	victim / 123456
Firewall	Disabled (ufw disabled)
Start Command	ip addr add 10.20.20.200/24 dev eth0; ip route add default via 10.20.20.1 dev eth0

5.3 Confirmed Attack Path

```
1 $ traceroute 10.20.20.200
2 1 10.202.91.125 13ms # Sarah's laptop (ZeroTier hop)
3 2 192.168.122.181 14ms # Alpine-Firewall-1 eth2
4 3 10.10.20.2 15ms # Router2
5 4 192.168.2.2 16ms # SwitchL32
6 5 10.20.20.200 16ms # Victim-1 *** TARGET REACHED ***
```

Listing 5.1: Traceroute Confirming Full 5-Hop Attack Path

5.4 Attack Results

5.4.1 Attack 1 — Network Reconnaissance (Nmap)

```

1 $ nmap -sV 10.20.20.200
2
3 PORT      STATE SERVICE VERSION
4 22/tcp    open  ssh      OpenSSH (Ubuntu)
5 80/tcp    open  httpd     Apache httpd 2.4.52 ((Ubuntu))

```

Listing 5.2: Nmap Scan Results

Result: SUCCESS — Two open services discovered: SSH on port 22 and HTTP on port 80.

5.4.2 Attack 2 — Web Server Access (curl + Nikto)

```

1 $ curl http://10.20.20.200
2 <title>Apache2 Ubuntu Default Page: It works</title>
3
4 $ nikto -h http://10.20.20.200
5 + Server: Apache/2.4.52 (Ubuntu)
6 + Missing X-Frame-Options header
7 + Missing X-Content-Type-Options header

```

Listing 5.3: HTTP Access and Vulnerability Scan

Result: SUCCESS — Apache 2.4.52 accessible with default page exposed, missing security headers.

5.4.3 Attack 3 — SSH Brute Force (Hydra)

```

1 $ hydra -l victim -P /usr/share/wordlists/rockyou.txt ssh
   ://10.20.20.200 -t 4
2
3 [22][ssh] host: 10.20.20.200 login: victim password: 123456
4 1 of 1 target successfully completed, 1 valid password found
5 Hydra finished at 2026-05-28 12:27:52

```

Listing 5.4: SSH Brute Force Results

Result: SUCCESS — Weak password cracked in **16 seconds** (attempt #1 in rockyou.txt).

5.4.4 Attack 4 — Post-Exploitation Recon

```

1 $ ssh victim@10.20.20.200 "id; uname -a; ip addr"
2 uid=1000(victim) gid=1000(victim) groups=1000(victim)
3 Linux Victim-1 6.19.11+kali-amd64 x86_64 GNU/Linux
4 inet 10.20.20.200/24 scope global eth0

```

Listing 5.5: Post-Exploitation via SSH

Result: SUCCESS — Full shell access confirmed, system information extracted.

5.4.5 Attack 5 — DoS SYN Flood (hping3)

```

1 $ sudo hping3 -S -p 22 --flood 10.20.20.200
2 HPING 10.20.20.200: S set, 40 headers + 0 data bytes
3 hping in flood mode, no replies will be shown
4 7,564,200 packets transmitted

```

Listing 5.6: DoS Attack

Result: SUCCESS — SYN flood executed, 7.5 million packets sent. Grafana showed traffic spike on monitoring dashboard.

5.5 Manual Testing Summary

Table 5.2: Manual Attack Results

Attack	Tool	Target	Port	Result	Duration
Reconnaissance	Nmap	10.20.20.200	Any	SUCCESS	7s
Web Access	curl	10.20.20.200	80	SUCCESS	1s
Web Scan	Nikto	10.20.20.200	80	SUCCESS	30s
SSH Brute Force	Hydra	10.20.20.200	22	SUCCESS	16s
Post-Exploit	SSH	10.20.20.200	22	SUCCESS	instant
DoS Flood	hping3	10.20.20.200	22	SUCCESS	ongoing

Key Finding: All five attack types succeeded against the undefended victim VM. The SSH password was cracked in only 16 seconds using the rockyou.txt wordlist, with 123456 as the very first attempt. The full attack path traverses 5 network hops across ZeroTier, the Alpine firewall, Router2, and SwitchL32 before reaching Victim-1. This establishes the baseline that the RL defender must learn to prevent.

Chapter 6

Reinforcement Learning Agents

6.1 Introduction to RL in Cybersecurity

Reinforcement Learning (RL) is a machine learning paradigm where autonomous agents learn optimal strategies through trial and error, receiving numerical rewards or penalties for every action they take within an environment. Unlike supervised learning, RL agents do not require labelled datasets — they learn entirely from the consequences of their own actions.

In this project, two RL agents compete inside a **live GNS3 network digital twin** in a **two-player zero-sum adversarial game**:

- **Red Agent (Attacker)**: Learns to maximise damage to the network by executing a diverse set of 18 cyber-attacks across four strategic phases (reconnaissance, intrusion, lateral movement, impact).
- **Blue Agent (Defender)**: Learns to minimise damage and maintain network availability using 18 defensive actions spanning immediate response, recovery, and proactive detection.

This adversarial self-play framework forces both agents to continuously adapt their strategies: as the Red agent discovers new attack sequences, the Blue agent must learn to counter them, and vice versa.

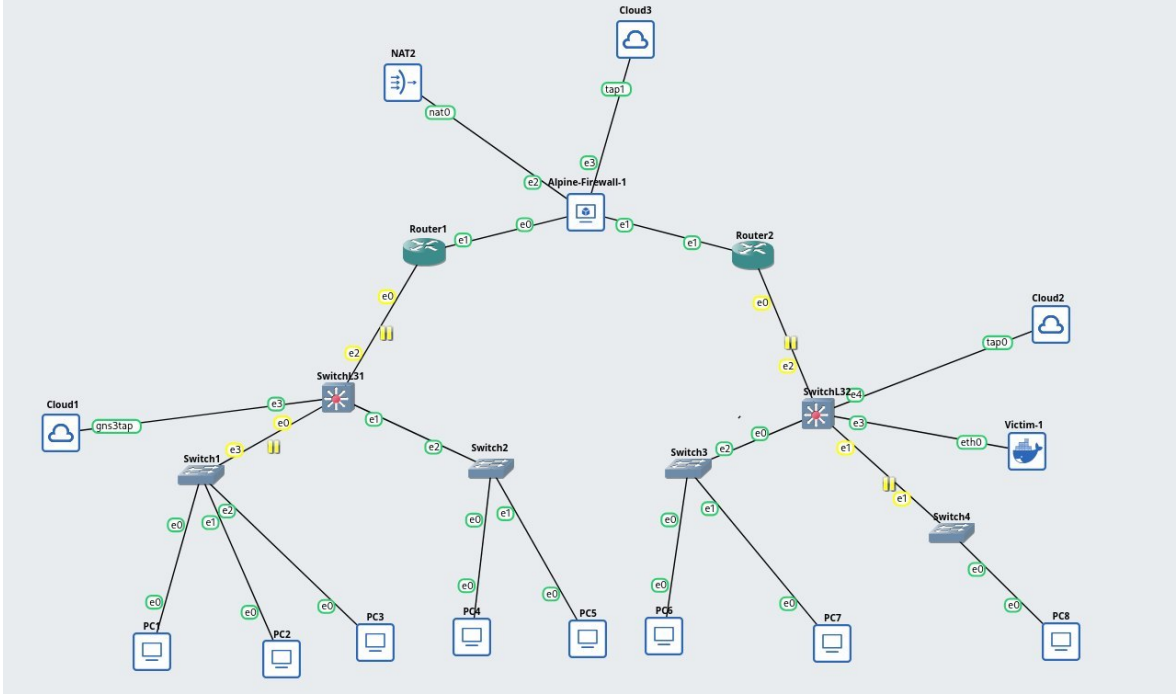


Figure 6.1: RL training in the topology

6.2 Environment Design

The RL environment follows the `gymnasium.Env` interface and is implemented across seven Python modules. Together, they connect the RL agents to the live GNS3 topology and provide the observation, action, and reward mechanics.

6.2.1 `gns3_connector.py` — Network Interface

Authenticates with the GNS3 REST API v3 using a JWT Bearer token and provides methods to:

- Fetch node lists and link lists from the running project.
- **Shut down** a link (suspend a GNS3 link between two nodes).
- **Restore** a suspended link.
- **Suspend** / **resume** entire nodes (simulating CPU exhaustion or DDoS).
- Push and retrieve running configurations (simulated via Telnet placeholders, ready for real deployment).

In **synthetic mode** (`TELEMETRY_MODE = "synthetic"`), all GNS3 API calls return immediate success without contacting the server, enabling rapid offline training.

6.2.2 `state_collector.py` — State Observation

Queries InfluxDB for real-time SNMP-collected network telemetry and constructs a normalised observation vector. Eight metric categories are collected per interface (see Section 6.3 for the full layout):

- Interface up/down status
- Log-scaled traffic rate (`in_octets`)
- Normalised error count (`in_errors`)
- CPU usage per device
- Packet loss percentage
- Broadcast traffic ratio (MAC flood indicator)
- ARP table size (ARP attack indicator)
- Anomaly score (deviation from normal baseline)

6.2.3 `attack_executor.py` — Attack Simulation Engine

Contains the `AttackExecutor` class with one handler method per Red-agent attack type (18 total). Each handler:

1. Perturbs the environment’s internal state arrays (traffic, CPU, broadcast ratio, anomaly score, etc.) according to the attack’s `severity` and `stealth` parameters.
2. For link-level attacks (Shutdown Link, Shutdown Core Link), makes real GNS3 API calls to suspend the physical link.
3. Registers the attack in the environment’s `active_attacks` tracking dictionary so the Blue agent’s defenses can counter it.

6.2.4 `defense_executor.py` — Defense Simulation Engine

Contains the `DefenseExecutor` class with one handler per Blue-agent defense type (18 total). Each handler:

1. Checks whether any currently-active Red attack matches its `counters` list.
2. If matched: reverses or dampens the state perturbations caused by the attack, clears the attack from the tracking dictionary, and returns success.
3. If no matching attack is active: the defense is a wasted move and the agent is penalised.

6.2.5 `environment.py` — Main RL Interface

Implements the `gymnasium.Env` interface and orchestrates the full simulation:

- **Observation space:** `Box(80,)` — continuous 80-dimensional vector.
- **Action space:** `Discrete(18)` — index into the attack or defense catalog.
- **Kill chain tracking:** Monitors Red’s progression through four stages (see Section 6.5).

- **Scenario selection:** Each episode begins with one of five starting conditions (see Section 6.7).
- **Episode metrics:** Tracks time-to-detect, time-to-recover, availability, false positives, and kill chain stage for thesis-ready logging.

6.3 State Space

The observation vector has **80 dimensions**, representing the live state of 10 monitored network interfaces across 8 metric groups.

Table 6.1: State Vector Layout ($10 \times 8 = 80$ dimensions)

Index Range	Metric	Scale	Description
[0, 10)	Interface Status	$\{0, 1\}$	1 = up, 0 = down
[10, 20)	Traffic Rate	$\log(1 + x)$	Log-scaled <code>in_octets</code>
[20, 30)	Error Count	[0, 10]	Normalised <code>in_errors</code>
[30, 40)	CPU Usage	[0, 1]	Fraction of CPU utilisation
[40, 50)	Packet Loss	[0, 1]	Fraction of packets lost
[50, 60)	Broadcast Ratio	[0, 1]	Broadcast pkts / total pkts
[60, 70)	ARP Table Size	[0, 1]	Normalised ARP entry count
[70, 80)	Anomaly Score	[0, 1]	Deviation from normal baseline

The last three metric groups are particularly important for attack detection:

- **Broadcast ratio** spikes during MAC flooding attacks (broadcast storm).
- **ARP table size** grows abnormally during ARP poisoning attacks.
- **Anomaly score** rises for any attack — but stealthy attacks (e.g., traffic sniffing) raise it minimally, while noisy attacks (e.g., DDoS) cause a sharp spike.

6.4 Action Space

Both agents have **18 discrete actions** each. Unlike the traditional approach where Red and Blue simply toggle the same interfaces on and off, our system implements **semantically distinct attack and defense types** that mirror real-world cybersecurity operations.

6.4.1 Red Agent — Attack Actions (18)

Table 6.2: Red Agent Attack Catalog

ID	Attack Name	Category	Target	Severity	Stealth
<i>Network Disruption</i>					
0	Shutdown Link	Disruption	Interface	0.60	0.70
1	MAC Flooding	Disruption	Switch	0.70	0.40
2	ARP Poisoning	Disruption	Switch	0.80	0.60
3	DDoS Simulation	Disruption	Router	0.75	0.30
4	CPU Exhaustion	Disruption	Device	0.65	0.50
5	VLAN Hopping	Disruption	Switch	0.75	0.70
6	STP Attack	Disruption	Switch	0.85	0.60
12	Shutdown Core Link	Disruption	Core	1.00	0.50
13	DDoS Core Router	Disruption	Core	1.00	0.30
<i>Reconnaissance</i>					
7	Port Scan	Recon	Network	0.20	0.80
8	Topology Mapping	Recon	Network	0.25	0.75
9	Traffic Sniffing	Recon	Interface	0.30	0.85
14	Credential Theft	Recon	Device	0.40	0.80
<i>Persistence</i>					
10	Backdoor Creation	Persistence	Device	0.90	0.70
11	Config Tampering	Persistence	Device	0.85	0.65
15	Lateral Movement	Persistence	Network	0.80	0.70
16	Firewall Bypass	Persistence	Firewall	0.95	0.65
17	VLAN Pivot	Persistence	Switch	0.85	0.70

Each attack has two key parameters:

- **Severity** (0–1): Controls how strongly the attack perturbs the environment’s state arrays (traffic, CPU, errors, etc.). Higher severity means more damage to the network.
- **Stealth** (0–1): Controls how much the attack raises the anomaly score. High-stealth attacks (e.g., traffic sniffing at 0.85) are nearly invisible; low-stealth attacks (e.g., DDoS at 0.30) cause an immediate spike in the anomaly score.

6.4.2 Blue Agent — Defense Actions (18)

Table 6.3: Blue Agent Defense Catalog

ID	Defense Name	Category	Counters (Red Attacks)
<i>Immediate Response</i>			
1	Port Isolation	Immediate	MAC Flooding, VLAN Hopping
2	MAC Blacklisting	Immediate	MAC Flooding, ARP Poisoning
3	Rate Limiting	Immediate	DDoS Simulation, DDoS Core Router
4	ICMP Blocking	Immediate	DDoS Simulation, DDoS Core Router
13	Firewall Rule Update	Immediate	Firewall Bypass, VLAN Pivot
14	STP Guard Activation	Immediate	STP Attack
15	ARP Inspection Enable	Immediate	ARP Poisoning, MAC Flooding
<i>Recovery</i>			
0	Restore Link	Recovery	Shutdown Link, Shutdown Core Link
5	Traffic Rerouting	Recovery	Shutdown Link, Shutdown Core Link, DDoS Core Router
6	VLAN Reassignment	Recovery	VLAN Hopping, VLAN Pivot
7	Config Restoration	Recovery	Config Tampering, Backdoor Creation
8	Failover Activation	Recovery	Shutdown Core Link, DDoS Core Router
17	Full Network Lockdown	Recovery	Config Tampering, Backdoor Creation, Lateral Movement
<i>Proactive / Intelligent</i>			
9	Honeypot Deployment	Proactive	Port Scan, Topology Mapping, Traffic Sniffing, Credential Theft
10	Anomaly Detection Alert	Proactive	MAC Flooding, ARP Poisoning, DDoS Simulation
11	Dynamic ACL	Proactive	VLAN Hopping, Lateral Movement, Firewall Bypass
12	Traffic Baselining	Proactive	Traffic Sniffing, DDoS Simulation
16	IDS Alert Response	Proactive	Port Scan, Topology Mapping, Lateral Movement

Each defense action specifies a **counters** list — the Red attack names it can neutralise. A defense action only succeeds if at least one of the attacks it counters is currently active in the environment. This forces the Blue agent to learn the correct **attack-defense mapping** rather than randomly applying defenses.

6.5 Kill Chain System

Real-world attackers follow a structured attack progression known as the **Cyber Kill Chain** [?]. Our system implements a four-stage kill chain that the Red agent must learn to execute sequentially:

Table 6.4: Kill Chain Stages

Stage	Phase	Actions that Advance this Stage
1	Reconnaissance	Port Scan, Topology Mapping, Traffic Sniffing, Credential Theft
2	Intrusion	ARP Poisoning, VLAN Hopping, MAC Flooding, STP Attack
3	Lateral Movement	Lateral Movement, VLAN Pivot, Firewall Bypass, CPU Exhaustion
4	Impact	Shutdown Core Link, DDoS Core Router, Config Tampering, Backdoor Creation, DDoS Simulation

Rules:

- Red must complete ≥ 2 reconnaissance actions before Stage 1 is reached.
- Each subsequent stage requires the previous stage to be completed first (e.g., intrusion only begins after recon is done).
- **Kill chain completion** (Stage = 4) grants the Red agent a massive +50 reward bonus and **immediately terminates the episode**.
- If the Blue agent clears all active attacks before Red reaches Stage 4, Blue receives a **chain interruption bonus** of +30.

This mechanism teaches the Red agent to chain actions strategically (scout first, then intrude, then spread, then strike) rather than randomly selecting attacks.

6.6 Reward Function

The reward function is the most critical component for guiding agent learning. Our system uses a **multi-factor reward function** that goes far beyond simple binary success/failure.

6.6.1 Red Agent Reward

Table 6.5: Red Agent Reward Components

Component	Value	Condition
Base reward	+5 to +20	Per attack type (ARP poison = +15, DDoS = +12, Port scan = +5, Shutdown core = +20)
Core multiplier	$\times 1.5\text{--}2.0$	Applied when hitting core infrastructure
Impact scaling	$\times (0.7 + 0.3 \cdot \frac{ \text{affected} }{N})$	More affected interfaces = more reward
Stealth bonus	$+5 \times \text{stealth}$	When anomaly score stays below 0.35
Kill chain bonus	+2 per stage	Later stages reward more
Kill chain complete	+50	All 4 stages completed in one episode
Honeypot penalty	-30	Red triggered a deployed honeypot
Failed action	-1	Attack did not succeed

6.6.2 Blue Agent Reward

Table 6.6: Blue Agent Reward Components

Component	Value	Condition
Base reward	+7 to +15	Per defense type
Full recovery bonus	$\times 2.0$	All interfaces up and no active attacks
Response speed bonus	+5 (1-step), +2.5 (≤ 3 -step)	Faster response = higher bonus
Proactive detection	+5	Blue detected attack before full impact
Chain interruption	+30	Blue stopped kill chain before Stage 4
Honeypot trap	+25	Red was caught by deployed honeypot
Wasted move	-2	Defense used when no matching attack was active
Core down penalty	-2 per step	While core infrastructure remains under attack

6.7 Scenario Variety

To prevent both agents from overfitting to a single starting condition, each episode begins with a randomly selected scenario:

Table 6.7: Episode Scenario Types

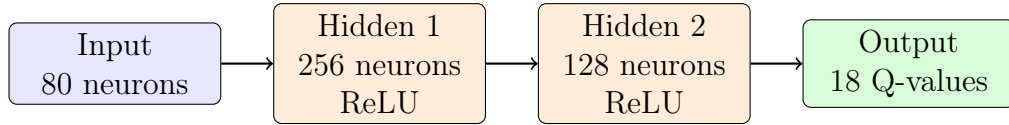
Scenario	Starting Conditions
Normal	All links up, clean state. Red starts from scratch.
Already Compromised	One random non-core interface is already down at episode start.
High Traffic	Network pre-loaded with elevated traffic and 50–85% CPU usage.
Partial Defense	Blue starts with a honeypot already deployed.
Stealth	Near-zero anomaly baseline — any attack immediately stands out. Red must use high-stealth actions.

6.8 Training Algorithm — Deep Q-Network (DQN)

Both agents use **Deep Q-Networks (DQN)** [?] with several enhancements for stable training in adversarial settings.

6.8.1 Neural Network Architecture

Both agents share the same two-hidden-layer architecture:

Figure 6.2: DQN Architecture: $80 \rightarrow 256 \rightarrow 128 \rightarrow 18$

- **Weight initialisation:** Kaiming Normal (He initialisation), optimised for ReLU activations.
- **Loss function:** Mean Squared Error (MSE) between predicted Q-values and Bellman target values.
- **Optimiser:** Adam with learning rate $\alpha = 10^{-3}$.
- **Gradient clipping:** Max norm of 1.0 to prevent exploding gradients.

6.8.2 Target Network

A frozen copy of the Q-network (target network) is used for computing Bellman targets, preventing oscillation during training:

$$y_i = r_i + \gamma \cdot \max_{a'} Q_{\text{target}}(s'_i, a'; \theta^-) \quad (6.1)$$

The target network θ^- is synchronised with the online network θ every 10 training steps (hard copy).

6.8.3 Exploration Strategy

Both agents use ε -**greedy** exploration with exponential decay:

$$\varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 \cdot d^t) \quad (6.2)$$

where $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.05$, and decay rate $d = 0.995$. This ensures the agents start with purely random exploration and gradually converge toward greedy exploitation.

6.8.4 Experience Replay

Red Agent — Standard Replay

The Red agent uses a standard circular replay buffer (size = 5000) with uniform random sampling of mini-batches (batch size = 64).

Blue Agent — Prioritised Replay

The Blue agent uses a **Prioritised Experience Replay** buffer that biases sampling toward transitions recorded during active attacks. Specifically:

- Transitions where the mean anomaly score (indices [70, 80) of the state vector) exceeds 0.30 are stored in a **priority ("hot") queue**.
- All other transitions go to the normal queue.
- Each mini-batch is composed of 60% samples from the hot queue and 40% from the normal queue (subject to availability).

This ensures the Blue agent is disproportionately trained on situations where the network is under attack, accelerating its ability to learn effective responses.

6.8.5 Hyperparameters Summary

Table 6.8: DQN Hyperparameters

Parameter	Value
State dimension	80
Action dimension	18
Hidden layer 1	256 neurons
Hidden layer 2	128 neurons
Learning rate α	10^{-3}
Discount factor γ	0.95
ε start	1.0
ε minimum	0.05
ε decay rate	0.995
Replay buffer size	5000
Mini-batch size	64
Target network sync	Every 10 training steps
Gradient clipping	Max norm 1.0
Episodes	300
Max steps per episode	20

6.9 Self-Play Training Loop

Training follows an alternating **Red-then-Blue** self-play loop where both agents interact with the same environment within each episode step:

```

1 env = NetworkEnvironment()
2 red_agent = RedAgent()
3 blue_agent = BlueAgent()
4
5 for episode in range(300):
6     state, info = env.reset()    # Random scenario selected
7
8     for step in range(20):
9         # Red attacks
10        red_action = red_agent.act(state)
11        next_state, red_reward, done = env.red_step(red_action)
12        red_agent.remember(state, red_action, red_reward, next_state)
13        red_agent.train()
14
15        if done: # Kill chain completed
16            break
17
18        # Blue defends
19        blue_action = blue_agent.act(next_state)
20        next_state2, blue_reward, done = env.blue_step(blue_action)
21        blue_agent.remember(next_state, blue_action, blue_reward,
22                             next_state2)
23        blue_agent.train()
24
25        state = next_state2
26        if done: # Full recovery achieved
27            break

```

Listing 6.1: Self-Play Training Loop (Simplified)

Key design decisions:

1. Red acts first each step, then Blue observes the post-attack state and responds. This simulates the real-world asymmetry where attackers have the initiative.
2. The episode terminates early if Red completes the full kill chain (Red wins) or if Blue clears all attacks and restores the network (Blue wins).
3. Both agents train after every step (online learning), not just at episode end.

6.10 Logging and Metrics

The training loop produces comprehensive, thesis-ready logging:

6.10.1 CSV Episode Log

Each row captures one full episode with the following columns:

Table 6.9: CSV Log Columns

Column	Description
episode	Episode number
scenario	Starting scenario type
red_reward / blue_reward	Cumulative reward per agent
epsilon	Current exploration rate
red_loss_avg / blue_loss_avg	Average training loss
red_wins / blue_wins	Running win count
kill_chain_stage_reached	Highest kill chain stage Red achieved (0–4)
chain_interrupted	Whether Blue stopped the kill chain (0/1)
time_to_detect	Steps before Blue first detected an active attack
time_to_recover	Steps before full network recovery
network_availability_pct	Percentage of steps with zero damage
false_positive_rate	Fraction of steps Blue acted without any attack active
honeypot_triggered	Whether Red was caught by a honeypot (0/1)
attack_types_used	JSON list of attack names used in this episode
defense_types_used	JSON list of defense names used
attack_success_counts	JSON dict: {attack_name: count}
defense_success_counts	JSON dict: {defense_name: count}

6.10.2 Per-Action Statistics (JSON)

A separate JSON file records the cumulative success rate of every individual attack and defense type across all episodes:

```

1 {
2   "attacks": {
3     "ARP Poisoning": {
4       "attempts": 142,
5       "successes": 139,
6       "success_rate": 0.9789
7     },
8     "Port Scan": {
9       "attempts": 201,
10      "successes": 198,
11      "success_rate": 0.9851
12    }
13  }
14 }
```

Listing 6.2: Example Attack Statistics Output

6.10.3 TensorBoard Integration

Real-time training curves are exported to TensorBoard with the following scalars:

- Reward/Red and Reward/Blue — per-episode rewards
- Epsilon — exploration decay curve
- Loss/Red and Loss/Blue — average DQN training loss
- Wins/Red and Wins/Blue — cumulative win counts
- KillChain/StageReached — kill chain progression per episode
- Network/Availability_pct — network health over time
- Blue/TimeToDetect and Blue/TimeToRecover — response metrics

6.11 System Architecture Summary

Figure 6.3 summarises the complete RL system architecture and the data flow between components.

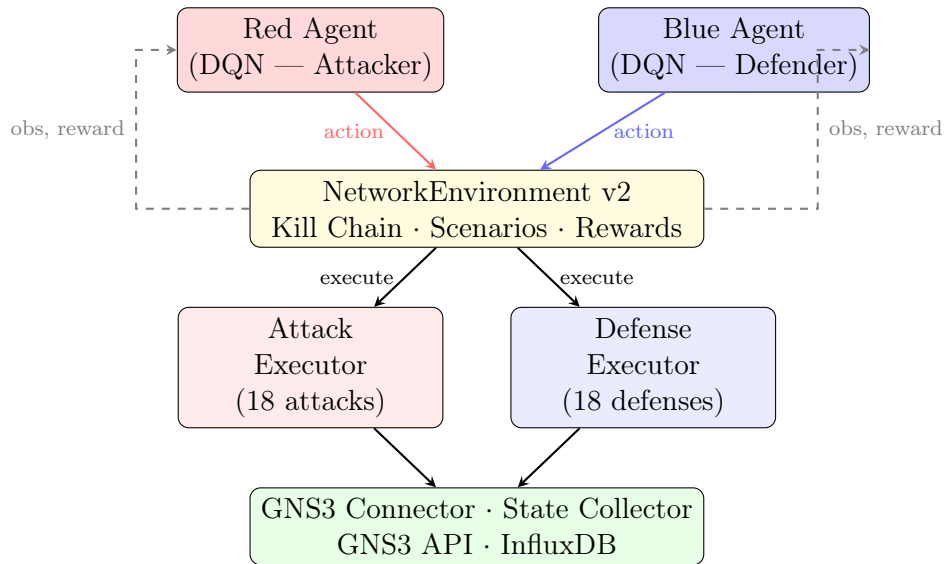


Figure 6.3: System Architecture: Red and Blue agents interact with the environment, which delegates to attack/defense executors backed by the live GNS3 network.

6.12 Summary

Table 6.10 compares the key features of our RL system against a baseline single-attack implementation.

Table 6.10: Feature Comparison: Baseline vs. Our Implementation

Feature	Baseline	Our System
Attack types	1 (link shutdown)	18 (disruption, recon, persistence)
Defense types	1 (link restore)	18 (immediate, recovery, proactive)
State dimensions	30	80
Action dimensions	8	18
Kill chain	None	4 stages
Scenarios	1 (normal)	5 types
Honeypot mechanism	None	Yes
Prioritised replay	No	Yes (Blue agent)
DQN hidden layers	128×64	256×128
Replay buffer	2000	5000
Metrics logged	8 columns	18 columns + JSON stats

This reinforcement learning system provides a realistic and comprehensive framework for training autonomous cybersecurity agents. By implementing 18 distinct attack types, 18 matching defenses, a four-stage kill chain, scenario diversity, and a multi-factor reward function, both agents develop sophisticated strategies that mirror real-world attack-and-defend dynamics within the live GNS3 network digital twin.

Chapter 7

Results and Conclusion

7.1 Project Achievements

Table 7.1: Project Deliverables Status

Component	Status	Details
GNS3 Topology (18 nodes)	Complete	3-tier, OSPF, 5 VLANs
Config Persistence	Complete	Startup configs in GNS3 project files
ZeroTier Collaboration	Complete	All 5 members connected
External Attack Path	Complete	Kali → ZeroTier → Firewall → Victim-1
NetBox Inventory (auto-sync)	Complete	GNS3 API → NetBox systemd service
Telegraf Collection	Complete	SNMP polling every 60s
InfluxDB Storage	Complete	Time-series metrics stored
Grafana Dashboard	Complete	5 security panels
Victim VM (SSH + Apache)	Complete	Docker, persistent start command
Manual Penetration Testing	Complete	5 attacks, all successful
RL Environment	Complete	3 Python files, live GNS3 integration
RL Agent Training	Complete	Training on live network

7.2 Key Technical Challenges

1. **GNS3 v3 API Migration:** The GNS3 server runs v3.0.5 with a new `/v3/` API prefix and OAuth2 authentication, requiring updates to all automation scripts from the documented v2 API.
2. **Config Persistence:** QEMU nodes lost NVRAM on restart because `write memory` was not sufficient — the fix was writing startup configs directly to GNS3 project filesystem files.
3. **Network Bridging:** Connecting the external Kali attacker to GNS3 required iptables rules on Sarah’s laptop (LIBVIRT_FWI chain) and on the Alpine firewall (eth2 → eth1 FORWARD + MASQUERADE).
4. **Static Route Conflicts:** Both routers had accumulated conflicting static routes that overrode OSPF learned routes, causing routing loops — identified via `show ip route` analysis and resolved by removing all redundant statics.

5. **Victim-1 Gateway:** Docker container start command had wrong gateway (10.20.20.254 instead of 10.20.20.1), fixed in GNS3 node configuration.
6. **SwitchL32 Port Discovery:** Victim-1 was connected to Gi0/3 (not the documented Gi1/0), discovered via CDP neighbor detail and ARP table analysis.

7.3 Limitations

- RL training requires GNS3 to be running — agents cannot train offline
- SNMP polling interval (60s) may miss rapid attack bursts
- Docker containers require explicit startup commands for IP persistence
- RL agents currently operate on interface-level actions only, not real attack tools
- ZeroTier connectivity is required for all team members to access the lab

7.4 Future Work

- Extend RL action space to include real attack tools (Nmap, Hydra, hping3)
- Add IDS (Snort/Suricata) alerts into the defender’s observation space
- Implement self-play training where agents improve against each other
- Add OPNsense firewall for more realistic multi-zone attack scenarios
- Integrate Syslog streaming from Alpine firewall for faster event detection
- Reduce SNMP polling interval to 10s for better attack burst detection

7.5 Conclusion

This project successfully demonstrates that a **Network Digital Twin combined with Adversarial RL** is a viable and powerful platform for autonomous cybersecurity research. Unlike purely simulated environments like CyberBattleSim, our agents interact with a **real virtual network** — making the learned strategies more applicable to real-world scenarios.

The manual penetration testing confirmed that the attack surface is properly configured (all 5 attacks succeeded across a real 5-hop network path), and the full monitoring stack (NetBox auto-sync, Telegraf, InfluxDB, Grafana) provides the observability needed for both human analysis and autonomous agent decision-making. The project overcame significant real-world integration challenges — API version migrations, routing conflicts, config persistence, and network bridging — making it a robust and reproducible platform.